

Code Explanation

DLT Pipeline Code Explanation

This code is designed to create a data pipeline using Databricks' Delta Live Tables (DLT) feature. It reads customer and order data from CSV files, applies data cleaning and transformations, and creates several tables for analysis.

Overview

The code defines a data pipeline that reads customer and order data, applies data cleaning and transformations, and creates several tables for analysis. The pipeline consists of four main tables: `customer`, `order`, `ordersummary`, and `customeraggregatespend`.

Step 1: Read Customer Data and Define Schema

The first step is to read the customer data from a CSV file and define its schema. The data is read using the `spark.read.csv` method, and the schema is defined explicitly using the `select` method.

```
df = spark.read.option("inferSchema", "true").csv(CUSTOMER_DATA_PATH, header=True)
df = df.select(
    F.col("CustId").cast("string"),
    F.col("Name").cast("string"),
    F.col("EmailId").cast("string"),
    F.col("Region").cast("string")
)
```

Step 2: Read Order Data and Define Schema

Similarly, the order data is read from a CSV file and its schema is defined.

```
df = spark.read.option("inferSchema", "true").csv(ORDER_DATA_PATH, header=True)
df = df.select(
    F.col("OrderId").cast("string"),
    F.col("ItemName").cast("string"),
    F.col("PricePerUnit").cast("double"),
    F.col("Qty").cast("int"),
    F.col("Date").cast("date"),
    F.col("CustId").cast("string")
)
```

Step 3: Add TotalAmount Column to Order Data

A new column `TotalAmount` is added to the order data by multiplying `PricePerUnit` and `Qty`.

```
df = df.withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
```

Step 4: Remove Nulls and Duplicates from Customer and Order Data

Nulls and duplicates are removed from both customer and order data using the `na.drop` and `dropDuplicates` methods.

```
df = df.na.drop().dropDuplicates()
```

Step 5: Create ordersummary Table

The `ordersummary` table is created by joining the `customer` and `order` tables on the `CustId` column.

```
customer_df = dlt.read("customer")
order_df = dlt.read("order")
joined_df = customer_df.join(
    order_df,
    on="CustId",
    how="inner"
).select(
    customer_df["CustId"],
    customer_df["Name"],
    customer_df["EmailId"],
    customer_df["Region"],
    order_df["OrderId"],
    order_df["ItemName"],
    order_df["PricePerUnit"],
    order_df["Qty"],
    order_df["Date"],
    order_df["TotalAmount"]
)
```

Step 6: Add SCD Type 2 Columns to ordersummary Table

SCD Type 2 columns (`IsActive`, `StartDate`, and `EndDate`) are added to the `ordersummary` table.

```
current_timestamp = F.current_timestamp()
joined_df = joined_df.withColumn("IsActive", F.lit(True))
joined_df = joined_df.withColumn("StartDate", current_timestamp)
joined_df = joined_df.withColumn("EndDate", F.lit(None).cast("timestamp"))
```

Step 7: Check if ordersummary Table Exists

The code checks if the `ordersummary` table already exists in the database.

```
tables = spark.sql(f"SHOW TABLES IN {CATALOG}.{SCHEMA}").filter(F.col("tableName") == "ordersummary").collect()
if len(tables) == 0:
    # First run - just return the joined data with SCD columns
    return joined_df
```

Step 8: Handle SCD Type 2 Logic for ordersummary Table

If the `ordersummary` table exists, the code applies SCD Type 2 logic to handle changes in customer data.

```
existing_df = dlt.read("ordersummary")
latest_existing = existing_df.withColumn(
    "row_num", F.row_number().over(window_spec)
).filter(F.col("row_num") == 1).drop("row_num")
changed_customers = customer_df.join(
    latest_existing.filter(F.col("IsActive") == True),
    on="CustId",
    how="inner"
).filter(
    (customer_df["Name"] != latest_existing["Name"]) |
    (customer_df["EmailId"] != latest_existing["EmailId"]) |
    (customer_df["Region"] != latest_existing["Region"])
).select(customer_df["CustId"]).distinct()
```

Step 9: Create customeraggregatespend Table

The `customeraggregatespend` table is created by aggregating the `TotalAmount` column by `Name` and `Date`.

```
ordersummary_df = dlt.read("ordersummary")
aggregated_df = ordersummary_df.groupBy("Name", "Date").agg(
    F.sum("TotalAmount").alias("TotalAmount")
)
```

Step 10: Return Aggregated Data

The aggregated data is returned as the `customeraggregatespend` table.

```
return aggregated_df
```